

Audio Matching Using Fourier Transform (FFT)

Project Overview

Have you ever used apps like “*Shazam*” to instantly recognize a song playing around you? With just a few seconds of audio, these systems can identify music from massive databases.

In this project, you will build a simplified version of such a system. Instead of matching against millions of songs, your task will be to locate a short audio clip within a longer recording. To achieve this, you will use the **Fast Fourier Transform (FFT)** to analyze audio signals in the frequency domain and compare their spectral patterns.

This project introduces you to the fundamental idea behind audio fingerprinting: **sounds that are similar share similar frequency characteristics.**



Project Goal

This project demonstrates how a theoretical concept like the **Fourier Transform** can be applied to solve a real-world problem. By completing it, you will gain practical intuition about how signals can be analyzed, compared, and matched using their frequency content.

By the end of this project, you should be able to:

- Work with real audio data in Python
- Convert signals from time domain to frequency domain using FFT
- Interpret frequency spectra

- Implement a sliding-window matching algorithm
- Design a similarity metric for signal comparison
- Visualize and evaluate detection results

Project Guidelines

- Groups of Three students maximum (from the same Lab group, **NO** cross-labs allowed)
- Copied reports or code means zero for both versions!
- You should submit a **hard copy** of:
 - Code: Your Python script/s
 - Report:
 - Time domain plots
 - Frequency domain plots
- The deadline to submit your Reports/Codes is on Monday May 11th to your corresponding lab instructor.
- Evaluations will take place during the lab sessions starting Tuesday May 12th. All team members must attend the evaluation (Evaluation is individual). You must bring a laptop to your evaluation.

Expected Outputs

Your program should print:

- Sampling frequency
- Length of full signal
- Clip length
- Original clip position
- Detected position
- Best similarity score

Plots:

- Full signal (time domain)
- Clip (time domain)
- Clip (frequency domain)
- Similarity score vs. time
- Clip vs. detected segment

Project Workflow Tasks

1. Load and Preprocess the Audio Signal

You will begin by reading a .wav file:

- Load the full audio signal and sampling frequency
- If the signal is stereo, convert it to mono
- Convert the signal to floating-point format for processing

Hint: `wavfile.read("audio_name.wav")`

2. Visualize the Signal in Time Domain

- Create a time axis using the sampling frequency
- Plot a small portion of the signal

3. Extract a Query Clip

- Select:
 - A starting time (in seconds)
 - A duration (in seconds)
- Convert these values into sample indices
- Extract the corresponding segment from the full signal

This segment represents the **query clip** that your system must detect.

4. Convert Signals to the Frequency Domain

Implement a function:

- Input: a signal segment
- Output: **one-sided FFT magnitude**

Apply this function to the query clip.

5. Visualize the Frequency Spectrum

- Create a frequency axis
- Plot the FFT magnitude of the clip

6. Perform Audio Matching

Implement a **sliding window algorithm**:

- Move a window across the full signal
- Window length = clip length
- Use a step size for efficiency. Note that the step size affects: Speed (larger = faster) and Accuracy (smaller = more precise)

For each windowed segment:

1. Compute its FFT
2. Compare it with the clip's FFT
3. Compute a similarity score (using the dot product)
4. Make sure to normalize the score (divide by product magnitudes) for fair comparison

Hint: `np.linalg.norm(A)` returns magnitude of vector A

7. Detect the Best Match

- Identify the segment with the **highest similarity score**
- Convert its index to time

This is your system's prediction of where the clip appears.

8. Visualize Matching Results

You should generate:

- A plot of similarity score vs. time
- A marker for:
 - The actual clip position
 - The detected position

9. Compare Signals

- Extract the detected matching segment
- Plot:
 - Original clip
 - Detected segment